Observability-Aware Trust-Based Grayhole Detection for Underwater Acoustic Networks

Team Members

241CS110, Appaji Nagaraja Dheeraj, appajidheeraj.241cs110@nitk.edu.in

241CS111, Aryan Bokolia, aryanbokolia.241cs111@nitk.edu.in

241CS117, Karthikeya Gupta, karthikeyagupta.241cs117@nitk.edu.in

Abstract

Underwater acoustic networks rely on relay nodes to carry sensor data to a surface sink, and a grayhole relay can secretly drop part of that traffic. The classic defence is a watchdog that listens for each forward and accuses a relay whose missed forwards exceed a fixed threshold. Underwater, however, forwards are also missed because of half-duplex deafness, collisions at the relay and long, lossy acoustic links, and these losses grow with traffic. A fixed watchdog therefore catches grayholes only by also accusing honest nodes, and routing around those honest nodes costs delivery and throughput. This work implements, in UnetStack, a network-layer detector that judges only windows the monitor could have observed, compares the forwards heard with the number an honest relay should show on each link given its distance and the normal loss of its area, combines monitors as an area-weighted trust value, and flags relays with a dynamic threshold and two-round confirmation. The sink receives the evidence as reports over the acoustic channel, gives the administrator a ranked suspect list and routes traffic around confirmed grayholes. In 12-hour simulations on ten test networks with one, two and three grayholes, the detector caught 95% of the grayholes while accusing 0.3 honest nodes per run, against 7.2 for the fixed watchdog, and gave the highest delivery ratio and throughput of the compared methods.

1 Introduction

1.1 Background and challenges

Underwater sensors often need intermediate relays to deliver measurements to a sink. Acoustic communication introduces propagation delay and half-duplex operation: a node cannot receive normally

while it transmits. UnetStack provides the protocol agents and modem notifications needed to study these interactions in simulation [5]. This project concerns network-layer forwarding; physical-layer events supply monitoring evidence.

A watchdog listens for a neighbour to forward a packet. Failure to overhear that forward has several possible causes. The neighbour may have deliberately dropped the packet, may never have received it, or may have forwarded it while the monitor failed to decode the transmission. The original watchdog literature already identifies collision-related limitations [1]. The problem is therefore not solved by assigning a trust score to every silence.

Several challenges affect evaluation as well as design. Filtering more windows can reduce false accusations simply by discarding evidence. Different monitors can observe the same packet, so their counters are correlated. A learned normal-loss estimate can itself be contaminated by attackers. Finally, a detector that performs well with complete offline evidence may perform differently when reports must cross a lossy network.

1.2 Problem statement and objectives

We investigate whether locally observable conflict checks and expected-loss correction can improve grayhole detection relative to explicitly defined baseline adaptations. The method must use monitor observations rather than attacker labels, and its performance must be assessed together with false accusations and reporting cost.

1. Implement and explain a watchdog that distinguishes retained forwarding windows from locally unobservable windows.

2. Compare the proposed detector, a fixed watchdog, a CAD-style adaptation and component variants on the same recorded traffic, with thresholds tuned on separate networks that are never used for testing.

3. Measure delivery, delay and transmission cost when detection, reporting and route response occur in time order.

The study does not aim to establish immunity to false reports, field performance, or superiority over complete implementations of every published protocol.

1.3 Organization

Section 2 positions the work and identifies the implemented comparators. Section 3 gives the model and algorithm. Section 4 defines the experiments and metrics. Section 5 reports detection and network outcomes. Section 6 discusses validity and limitations, followed by the conclusion in Section 7.

2 Literature Review

2.1 Watchdog monitoring

Marti et al. introduced watchdog monitoring together with pathrater-based routing [1]. The monitor retains packet information and checks for a matching retransmission. The paper discusses ambiguous collisions, receiver collisions and partial dropping among its limitations. Its advantage is that forwarding can be checked through overhearing; its limitation is that silence does not uniquely identify the cause of loss. Our fixed-rate watchdog is an inspired comparator, not a reproduction of the full watchdog/pathrater system.

2.2 Channel-aware detection

Shila et al. estimate normal losses caused by channel impairment and medium-access collisions, then compare measured loss with the normal-loss estimate [2]. Their published CAD method includes upstream and downstream monitoring and adaptive detection thresholds. This establishes that channel correction and adaptive thresholds are existing ideas. Our CAD-style comparator retains a downstream channel/collision correction using monitor busy time. It omits the published upstream protocol, probe/acknowledgement exchange and signed reports. Its performance therefore cannot establish a win over the complete published CAD algorithm.

2.3 Selective forwarding in UnetStack

Chandavarkar and Suresh demonstrate selective forwarding and a detection mechanism in UnetStack3 [3]. The supplied implementation description uses destination-side timeouts and forwarding counters. It motivates studying the attack in an underwater networking stack, but is not the sender-overhearing comparator used in our experiments. We extend the project question toward local observability, expected loss and a ranked suspect list rather than claiming to reproduce that paper.

2.4 Underwater trust-aware routing

Han et al. combine trust evaluation and fuzzy logic with layered underwater routing [4]. Their model addresses forwarding selection using trust and neighbourhood information. This provides relevant context for using trust in an underwater network, but does not itself validate our observation filter or formulas. We do not implement their routing protocol as an experimental baseline.

2.5 Comparison and scope of the contribution

Approach	Useful idea	Boundary of this comparison
Watchdog [1]	Overhear forwarding	Our comparator uses a fixed miss-rate threshold, not full pathrater.
CAD [2]	Estimate channel and collision loss	Our downstream adaptation is not full CAD.
UnetStack3 study [3]	Demonstrate selective forwarding	Destination-timeout method is not reproduced here.
Trust-aware routing [4]	Use trust in forwarding selection	Background work, not a measured comparator.
Proposed method	Filter local conflicts; pool expected-forward evidence	Known positions/channel curve; honest reports; simple static simulation.

Table 1: Positioning of the project. The table does not establish that no earlier method uses related ideas.

The specific contribution evaluated here is the combination of stated per-window exclusions, distance-informed expected forwards and a dynamic decision rule in this UnetStack setting. Component comparisons test whether the exclusions add value to the same estimator. A comprehensive novelty assessment remains separate from demonstrating this project-level improvement.

3 Proposed Approach

3.1 Baseline watchdog and our modifications

The baseline is the classic watchdog used to demonstrate selective-forwarding detection in UnetStack3. Each sender acts as a monitor of its next hop and follows two rules:

Rule 1: after sending a packet to the next hop, listen for its forward; a forward not heard in time is a miss.

Rule 2: if the fraction of misses exceeds a fixed threshold (30%), accuse the next hop.

Underwater, a forward can be missed although the relay is honest: the monitor itself was transmitting (half-duplex), the packet collided at the relay, or the long acoustic link lost it. These losses grow with traffic, so the fixed threshold accuses honest relays, and routing around them forces detours and lowers delivery. Taking into account what an acoustic node can actually observe, the following modifications were made:

- **Observability:** a window counts only if the monitor could have heard the forward; hidden windows are ignored instead of being counted as misses.
- **Trust from each link's expected loss:** expected forwards are computed per link from its distance and the normal loss of its distance area (near, medium, far), and trust is heard divided by expected forwards, combined over all monitors as an area-weighted average.
- **Evaluation with a dynamic threshold and confirmation:** a relay is flagged only when

its shortfall is significant for the current network, and becomes a suspect only after two flagged rounds in a row; confirmed grayholes leave the baseline so further grayholes stand out.

Figure 1 contrasts the two decision paths.

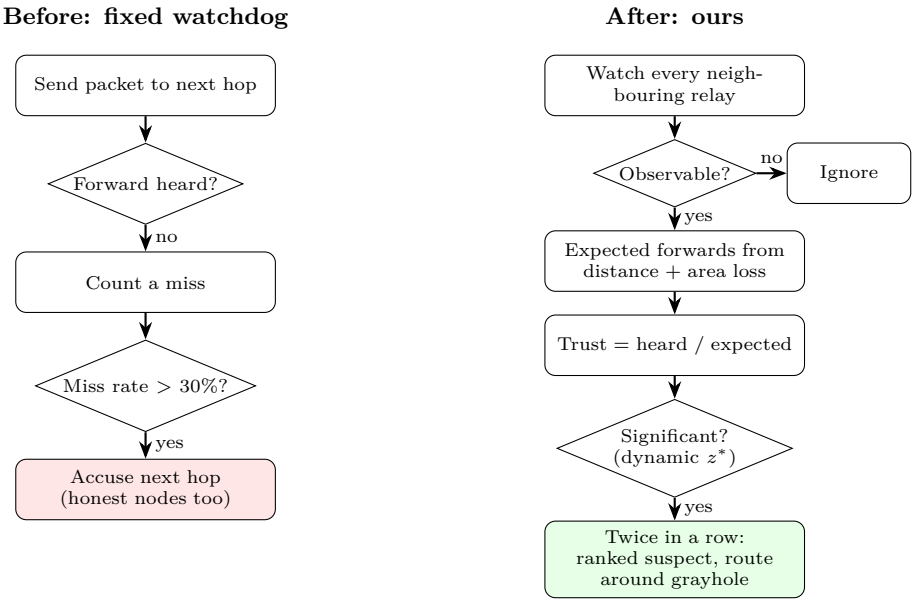


Figure 1: Before and after: the baseline watchdog's decision path and ours.

3.2 System and attacker model

The standard topology contains 20 static sensors and one sink. Positions and a distance-loss curve are assumed known. All monitors report their counters truthfully, including nodes that maliciously drop relayed packets. An attacker drops each received relayed packet independently with probability p; its own source traffic is not attacked. Several attackers mean independently dropping nodes, not coordinated collusion. Ground-truth attacker identities are used only for experiment construction and scoring.

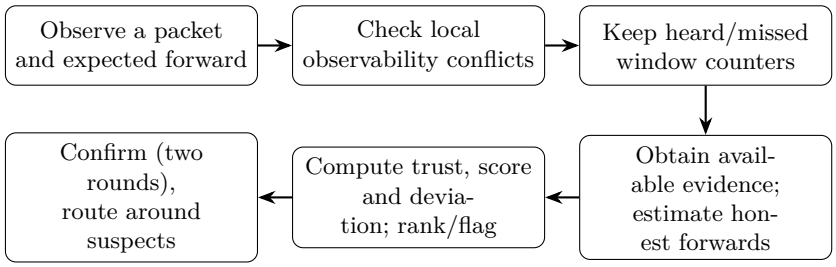


Figure 2: Implemented flow. In the live network the sink sees only reports that actually arrive over the acoustic channel, plus its own observations. An excluded window contributes neither a heard nor a missed count.

3.3 Observability checks

For monitor M, sender A and relay B, known distances and sound speed estimate the packet's arrival interval at B and the interval in which M expects a forward. The no-MAC forwarding allowance is 2 s; the CSMA scenario uses 10 s. These are configured observation allowances, not guaranteed queueing bounds.

The main rule retains a window only when (C1) the monitor did not transmit during its watch interval, (C3) the monitor did not observe the relay transmitting during the incoming packet's arrival, and (C4) no transmission known to the monitor would overlap that arrival at the relay. These conditions apply to heard forwards and misses alike. Hidden interference and decoding losses remain possible.

A corrupted-frame notification at the monitor (C2) is not an exclusion in the main method. The relay's own attempted forward can cause that notification, while deliberate dropping produces no frame. Excluding it can preferentially remove honest forwarding attempts. Applying C1+C3+C4 is not a proof of unbiased sampling; the remaining selection dependencies require empirical checks.

3.4 Expected forwards and score

Let the channel loss at distance d be $e(d) = 0.15(d/1500)^2$ within range. For retained window w, the channel-only probability of hearing an honest forward is approximated by

$$g_w = (1 - e(d_{AB}))(1 - e(d_{BM})). \tag{1}$$

For each monitor–relay link i, let n_i be retained windows, O_i heard forwards and $G_i = \sum_w g_w$. Its residual loss is

$$\kappa_i = \max(0, 1 - O_i/G_i). \tag{2}$$

Links with at least five windows and positive G_i enter the estimator. Near, medium and far groups are distance tertiles. For a judged relay B, each group's residual loss is the median over other relays' links. If no alternative exists, the group-wide fallback includes its own links. This fallback loses leave-self-out protection.

$$E_i = G_i(1 - \kappa_{\text{area},-B}), \qquad T_B = \frac{\sum_i O_i}{\sum_i E_i}, \qquad S_B = 100 \max(0, 1 - T_B). \tag{3}$$

If the honest expectation were correct and an independent drop process retained fraction $1 - p$, the expected observed count would be $(1 - p)E$. This motivates $1 - O/E$ as a drop estimate. In practice E is estimated, so the formula is model-based rather than a universal unbiased estimator. Zero expected forwards make a relay unjudgeable. Trust may exceed one; the suspiciousness score is then zero.

3.5 Dynamic threshold and confirmation

The raw deviation uses

$$z_B = \frac{\sum_i E_i - \sum_i O_i}{\sqrt{\sum_i E_i \max(0.01, 1 - E_i/n_i)}}. \tag{4}$$

A scale factor $\phi = \max(1, \text{median}(z_B^2)/0.455)$ is computed over judged relays with negative raw z. With fewer than two such relays, all judged relays are used. Each deviation is divided by $\sqrt{\phi}$. For k judged relays, the numerical threshold is $z^* = \Phi^{-1}(1 - \alpha/k)$. The relay is flagged only with at least 20 windows, $S_B \geq 10$, and corrected $z_B > z^*$.

The expectation, dispersion and number of judged relays can change with the observations. Alpha, the minimum count and the score floor are fixed within an experiment. Correlation between monitors means neither the nominal normal approximation nor its Bonferroni-shaped boundary guarantees a false-alarm probability. We therefore choose alpha on separate tuning networks (topologies 101–106), never on the test networks.

Several grayholes: baseline cleaning. With two or three busy grayholes in one area, the area's normal loss rises, honest relays look better than expected, ϕ grows to 50–300 and hides every grayhole, although the grayholes still rank first. The detector therefore removes relays with $S_B \geq 10$, and relays already confirmed as suspects, from the area medians and repeats the estimate (at most three times, until stable). On the calibration networks on the tuning networks this raised three-grayhole detection from 4/18 to 13/18 and two-grayhole detection from 9/12 to 12/12.

Confirmation. A relay becomes a suspect only when it is flagged in two consecutive 30-minute rounds, so a single noisy round accuses nobody; a confirmed suspect stays one until the administrator reviews it. The same rule is applied to every compared method.

3.6 Readable algorithm and implementation

> **At a monitor:** open a watch when a packet is sent or overheard toward a relay; record whether its forward is heard; close the watch; assign the outcome and conflicts to a counter bin.
>
> **At a decision:** select eligible counters; calculate channel expectation and area residual losses; calculate each relay's expected forwards and score; correct deviations for empirical dispersion; rank by score and apply the three alarm conditions.
>
> **During response:** every 30 minutes each monitor sends one bundled report; the sink keeps the latest delivered report of every link, runs the detector, confirms suspects over two rounds, gives the administrator the ranked list and routes traffic around the suspects wherever another path exists.

`MonitorNode.groovy` handles observations and reports; `Detectors.groovy` handles scoring; `Topology.groovy` handles routes. One pipeline (`run.sh`) calls these shared classes: `record.groovy` records evidence, `compare.groovy` replays the sink's decisions for every method on the same evidence, `network.groovy` runs the live network, and `plots.py` draws the graphs. The 32 counter

bins preserve outcome, sender/overhear status and conflict bits so component variants can reuse identical evidence.

A report is one data frame: a 12-byte header (packet header and busy fraction) and one 17-byte entry per watched relay (identifier, two 32-bit counters, 64-bit expected-forward sum), at most six entries in the 128-byte frame. Reports are jittered over ten minutes after each 30-minute checkpoint and traverse the same lossy network as data. The controller does not read remote agent memory. Route distribution itself remains instantaneous and uncharged.

4 Experimental Method

All experiments run in one pipeline (`run.sh`). `record.groovy` simulates the networks and stores every monitor's counters; `compare.groovy` replays the sink's 30-minute decisions for every method on exactly the same counters; `network.groovy` runs the live network, in which reports travel over the acoustic channel, the sink detects and confirms suspects and routes traffic around them; `plots.py` draws the graphs. The fixed watchdog and the CAD-style detector use the same reporting, confirmation and response as ours, so only the detection logic differs.

Parameter	Value
Simulator	UnetStack 7.0.0, protocol channel model with distance-dependent loss, Java 8
Network	20 sensors and one sink in 4 km × 4 km; range 1500 m; sound speed 1500 m/s; 40-sensor and sparse variants
Traffic	0.001–0.01 packets/s per sensor (detection); 0.005 and 0.01 (live network)
Grayholes	The 1, 2 or 3 busiest relays, each dropping 30% of relayed packets (drop-rate sweep 10–100%)
Networks	Tuning: topologies 101–106 (thresholds only); test: topologies 201–210
Duration	6 h recorded runs; 12 h live-network runs; one decision every 30 min
Own rules	Fixed watchdog miss rate $> 30\%$; CAD-style $z > 2$; ours $\alpha = 10^{-4}$
Same-budget thresholds	At most 5% of honest relays accused on the no-attack tuning runs: fixed 0.6, CAD-style $z > 2.5$, ours $\alpha = 10^{-4}$
Response	Confirmed suspects stop relaying wherever another path exists

Table 2: Simulation parameters.

Detection metrics are the fraction of grayholes confirmed by the end of a run and the honest relays confirmed as suspects. Network metrics are the delivery ratio and throughput (honest data packets delivered per hour) including all report traffic. A topology is the independent unit; intervals are 95% t-intervals over the ten test topologies, paired by topology where two methods are compared.

5 Results and Discussion

5.1 Before and after

Figure 3 summarises the whole evaluation. Under attack, the fixed watchdog catches every grayhole only by accusing 7.2 honest nodes per run, and routing around them costs most of what it gains: delivery rises only from 45.6% to 47.0%. CAD-style accuses fewer honest nodes (0.8) but misses 42% of the grayholes. Ours catches 95% of the grayholes while accusing 0.3 honest nodes per run, and gives the highest delivery ratio (49.7%) and throughput (200.5 against 186.8 packets per hour without defence).

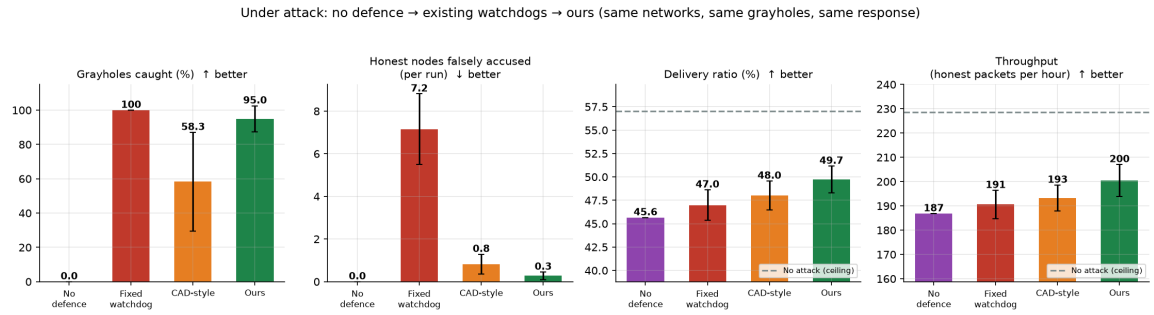


Figure 3: Before and after in the live network: grayholes caught, honest nodes accused, delivery ratio and throughput (12 h, ten test topologies, 1 grayhole at two loads and 2 and 3 grayholes pooled). The dashed line is the same networks without attack.

5.2 Why the fixed watchdog accuses honest nodes

Figure 4 shows honest relays confirmed as suspects as traffic grows. With no attacker at all, the fixed watchdog accuses 1.0 honest relay per run at the lowest load and 6.2 at the highest, because collisions and half-duplex deafness grow with traffic and are counted as drops. CAD-style accuses 0.3–0.7. Ours accuses 0–0.1, because unobservable windows are ignored and each link is compared with the loss it is expected to show.

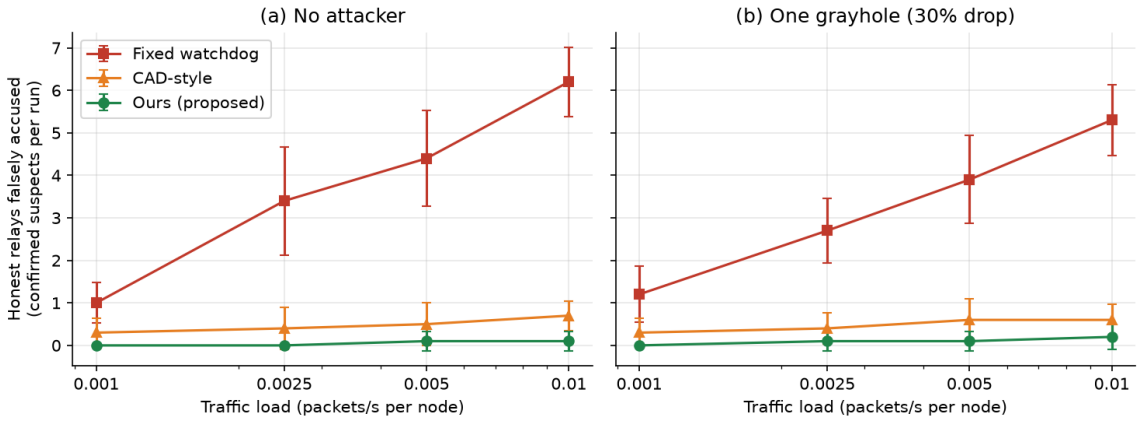


Figure 4: Honest relays falsely accused per run against traffic load, each method with its own rule (6 h, ten test topologies).

5.3 Grayhole detection in every scenario

Figure 5(a) is the fair contest: every method's threshold is tuned on the separate tuning networks so that at most 5% of honest relays are accused without an attacker. Ours detects 90% of single grayholes over four loads (CAD-style 58%, fixed 22%), 90% with traffic hotspots, 100% in the 40-node network, 95% with two grayholes (65%, 40%) and 77% with three (77%, 40%). Figure 5(b) shows the same methods with their own rules: the fixed watchdog reaches 100% detection only by accusing 41–71% of honest relays. Ours accuses no honest relay in the two- and three-grayhole cases. In the hotspot and 40-node networks ours accuses about 10% of honest relays, more than the fixed watchdog at the same budget.

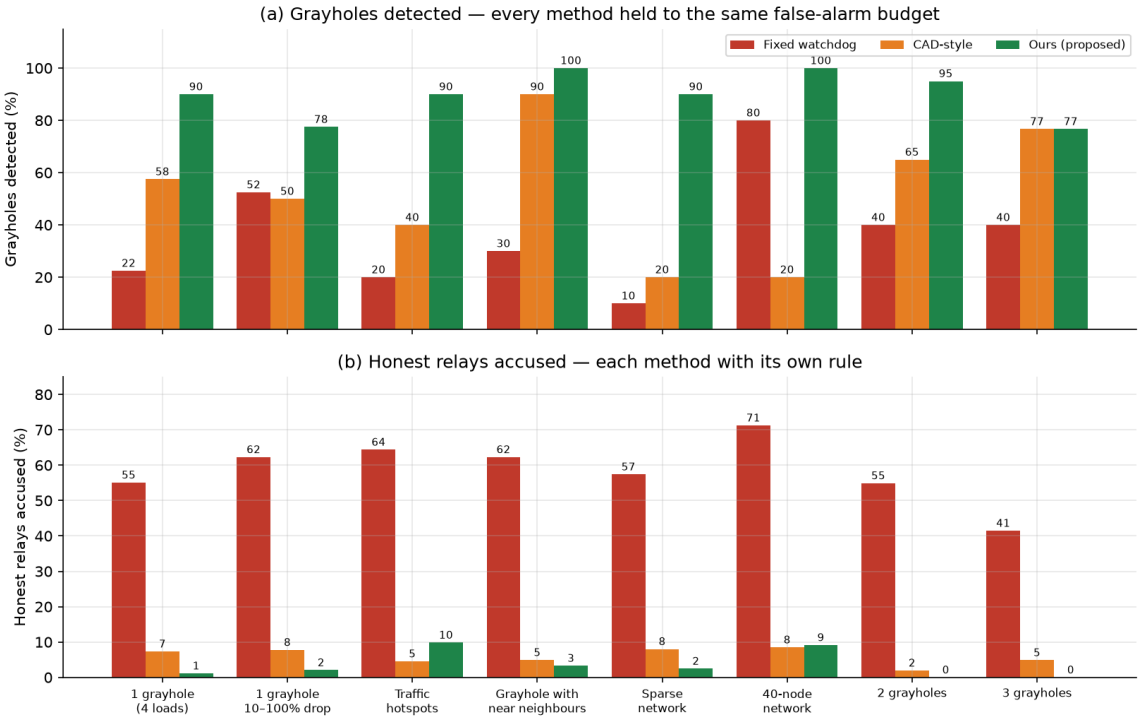


Figure 5: (a) Grayholes detected with all methods held to the same false-alarm budget; (b) honest relays accused with each method's own rule.

5.4 Ranked suspect list

Figure 6(a) is the list the administrator receives for a network with three grayholes. The three grayholes rank first with estimated drop rates of 38.9%, 32.1% and 28.2% and are confirmed as suspects; every honest relay is estimated at 0–0.2%. Figure 6(b) shows that the estimate follows the true drop rate (7.2, 18.5 and 47.3% for 10, 20 and 50%), while CAD-style underestimates moderate dropping.

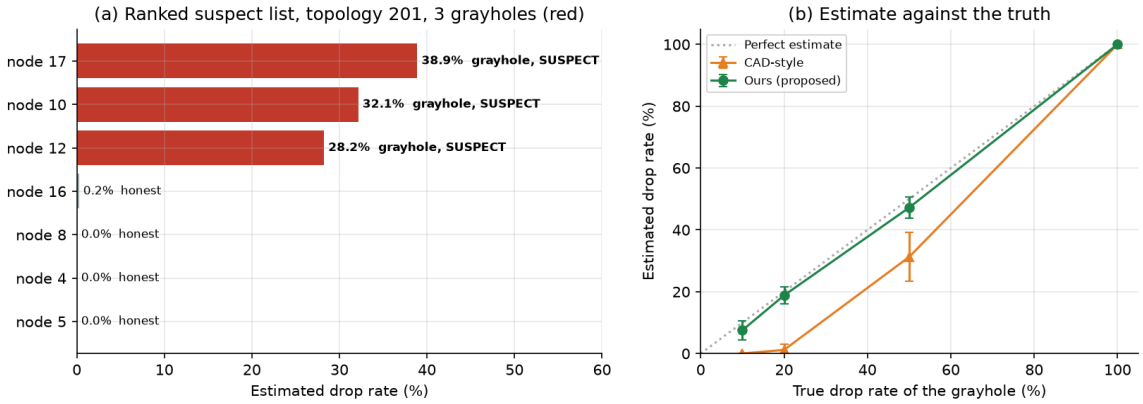


Figure 6: (a) Ranked suspect list for one test network with three grayholes; (b) estimated against true drop rate.

5.5 Network performance

Figure 7(a) breaks the live-network delivery down by setting. Ours has the highest delivery in all four. Compared with no defence on the same topologies it gains 4.0, 1.7, 5.6 and 4.9 percentage points (significant except at the higher load), and compared with the fixed watchdog 4.0 points with two and with three grayholes. Against CAD-style ours is ahead in every setting, but by 0.7–2.7 points, within the uncertainty. Figure 7(b) shows the mechanism over time with three grayholes: delivery falls when the attack starts and climbs back once ours has confirmed the grayholes. Figure 7(c) repeats the experiment with suspects blacklisted instead of avoided: every honest node the fixed watchdog accuses is then lost with the nodes behind it, and its delivery collapses to 23.2% against 45.6% without defence. Blacklisting also hurts a correct detector when a grayhole is the only path for many nodes, which is why routing around suspects is the response used here.

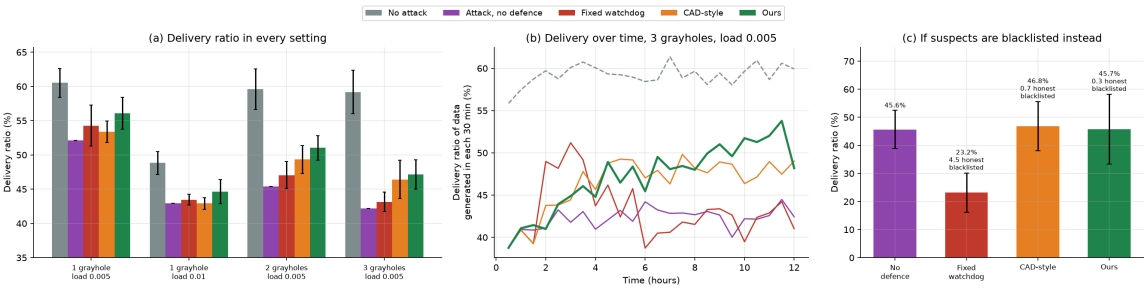


Figure 7: Live network: (a) delivery ratio per setting, error bars are paired 95% intervals against no defence; (b) delivery over time with three grayholes; (c) delivery when suspects are blacklisted.

5.6 Contribution of each part

Removing one part at a time, at the same 5% budget: without baseline cleaning, three-grayhole detection falls from 23/30 to 17/30; using an area average instead of the median gives similar detection (23/30); without the observability filter, detection is similar (37/40 single, 22/30 three grayholes) but honest accusations rise slightly (1.5% against 1.1% in the load sweep). With two-round confirmation in place, the filter's contribution is therefore small in these networks; its main effect is visible in the false accusations of Figure 4, where the methods without it count unobservable windows as drops.

6 Limitations and Reproducibility

1. Static connected topologies, known positions, a known distance–loss curve and truthful monitors; mobility, location error and falsified reports are not evaluated.

2. The CAD-style comparator is an adaptation, and the fixed watchdog follows the published rule, not every detail of its implementation.

3. Ten test topologies per setting; several comparisons are exploratory. Delivery gains over CAD-style are within the uncertainty.

4. Route updates are instantaneous and free in the simulation; report traffic, delay and loss are simulated and counted.

`./run.sh` regenerates every number and graph from the recorded counters in about an hour; `./run.sh demo` runs one network with three grayholes in about three minutes and prints the ranked suspect list; `./run.sh record` also re-records the counters.

7 Conclusion and Future Work

We implemented an observability-aware, trust-based grayhole detector for underwater acoustic networks in UnetStack and compared it with a fixed watchdog and a CAD-style detector in one pipeline. Judging only observable windows against each link's expected loss, with a dynamic threshold and two-round confirmation, the detector caught 95% of grayholes while accusing 0.3 honest nodes per run, against 7.2 for the fixed watchdog, and gave the highest delivery ratio and throughput in every live-network setting. The administrator receives a ranked list in which the grayholes come first. Future work should test dishonest monitors, mobility and location errors, larger and denser networks, where ours still accuses about 10% of honest relays, and a faithful implementation of the published comparators.

Author review note

This draft uses AI-assisted code review and manuscript preparation. The team must verify its contents, understand the implementation, and apply the disclosure and authorship rules of the actual submission venue. The supplied article layout is a formatting reference, not confirmation of journal acceptance or venue compliance.

References

[1] S. Marti, T. J. Giuli, K. Lai and M. Baker, "Mitigating routing misbehavior in mobile ad hoc networks," in *Proc. ACM MobiCom*, pp. 255–265, 2000. doi: 10.1145/345910.345955.

[2] D. M. Shila, Y. Cheng and T. Anjali, "Mitigating selective forwarding attacks with a channel-aware approach in WMNs," *IEEE Transactions on Wireless Communications*, vol. 9, no. 5, pp. 1661–1675, 2010. doi: 10.1109/TWC.2010.05.090700.

[3] B. R. Chandavarkar and N. Suresh, "Demonstration of selective forwarding attack and detection mechanism in UnetStack3," in *Proc. ICCCNT*, pp. 1–6, 2021. doi: 10.1109/ICCCNT51525.2021.9580125.

[4] D. Han, X. Du, L. Wang, X. Liu and X. Tian, "Trust-aware and fuzzy logic-based reliable layering routing protocol for underwater acoustic networks," *Sensors*, vol. 23, no. 23, art. 9323, 2023. doi: 10.3390/s23239323.

[5] UnetStack, *UnetStack Handbook*, "Physical," `https://unetstack.net/handbook/305_phy.html`, accessed 12 October 2026.